

3. Принятые решения, включая отброшенные

Для каждого решения указано: что выбрано, что рассматривалось и отброшено, почему, какой метод получился в итоге и где он описан или реализован. Номера (P-N) используются в остальных документах.

Первичные журналы решений, из которых собран этот перечень:

- проектный план — [Think-Faster: tasks/plan.md §4, §10](#);
- журнал исследования модели — [ML/results/analytics.md](#), таблица статусов в начале;
- вводные по модели — [ML/вводные.md](#);
- особые решения — [docs/документация.md §5.6, §6](#);
- решения BFF — [think-bff: BFF/docs/DECISIONS.md](#);
- инфраструктура — история коммитов [think-infra@dev](#) и [SYSTEM.md, Часть VI](#).

3.1. Контур и архитектура

№	Решение	Отброшено	Почему	Итоговый метод	Где
P-1	Сервисный контур: отдельные сервисы за одним nginx	монолит на FastAPI (план 16.09)	разные языки и владельцы частей, независимая выкатка, требование ТЗ §14 «решение — отдельный сервис или компонент»	9 контейнеров приложений + инфраструктура в одной docker-сети	SYSTEM.md 1.2–1.3
P-2	Без сервиса <code>dispatch</code>	<code>dispatch</code> разводит тревоги в BFF, почту и Telegram (схема 18.09)	адреса дежурных и заявки живут в BFF, промежуточный сервис дублировал бы данные	BFF сам создаёт заявку и публикует уведомление по факту	BFF DECISIONS.md «Событие по факту»
P-3	Уведомления — через очередь RabbitMQ	SMTP прямо из BFF; HTTP-ручки сервиса уведомлений с JWT	повторы, DLQ и дедуп без потери писем, BFF не ждёт почтовый сервер	<code>tf.notifications</code> → <code>tf-mail</code> / <code>tf-tg</code>	BFF DECISIONS.md «Пересмотр... письма через RabbitMQ» , think-infra: mailing/
P-4	Почта и Telegram — два сервиса	один <code>tf-notify</code> на два канала (заготовка docs/backend/notify)	независимые отказы (почта без Telegram и наоборот), свои учётки брокера и секреты	<code>tf-mail</code> , <code>tf-tg</code> ; общий модуль очереди	think-infra: mailing/app/broke r.py
P-5	Kafka для потока, RabbitMQ для команд и уведомлений	всё в одном брокере	поток показаний — журнал с хранением и повторным чтением (Kafka); команды и письма — ровно одному получателю с подтверждением (RabbitMQ)	топики <code>tf.ingest.*</code> , <code>tf.forecast.results</code> ; очереди <code>tf.model.commands</code> , <code>tf.notify.*</code>	kafka/topics.conf , rabbitmq/definitions.json

№	Решение	Отброшено	Почему	Итоговый метод	Где
P-6	Приём потока — HTTP-ручка воронки, дальше Kafka	шина пишет прямо в Kafka; отдельный analyzer	шине проще HTTPS с токеном; воронка проверяет события и отмечает молчание каналов	POST /api/funnel/events → tf.ingest.*	docs/backend/funnel/
P-7	Живые показания фронту — WebSocket воронки	WebSocket через dispatch или BFF	данные уже в воронке, без лишнего перехода	WSS /api/funnel/stream	funnel/stream.go
P-8	Одна база PostgreSQL, по схеме на сервис	два контейнера (аудит отдельно, план 18.09)	один сервер, одна выкатка и бэкап; изоляция достигается ролями схем	tf-postgres, схемы auth, bff, audit, роли _admin/_user	postgres/
P-9	Оперативный контур, архив и витрины — вне PostgreSQL	схемы ops/archive/marts с 313 млн строк в PostgreSQL	ретропрогоны в PostgreSQL заняли бы часы, в DuckDB — секунды; модели нужен только горячий журнал	горячий журнал DuckDB (100 суток), архив — файлы, витрина — Parquet	02. Архитектура 2.5
P-10	Версии модели — пакет в образе и manifest.json	MLflow, feature store, Airflow, Kubernetes	избыточны для одного сервиса модели и одного сервера	пакет модели (bundle.py), переключение командой model.switch	ML/INTEGRATION.md §6, §13.6
P-11	Собственные часы такта модели	APScheduler, Celery	такт должен переживать перезапуск и не повторяться (раньше было 120 сообщений в час)	clock.py: граница часа MCK + 120 с, отметка в clock.json	ML/service/clock.py
P-12	Фронт на Create React App и React 19, окна по правам	React 18 + Vite; MapLibre и ECharts из плана	сделано в CRA, внешние библиотеки карты и графиков не понадобились	сетка окон, окно видно по правам	app/ARCHITECTURE.md

3.2. Безопасность и доступ

№	Решение	Отброшено	Почему	Итоговый метод	Где
P-13	JWT RS256, access 10 мин + refresh 24 ч в HttpOnly-cookie	сессии на сервере; токен в localStorage	проверка в каждом сервисе без похода в tf-auth; cookie недоступна скриптам	access_token, refresh_token	TokenService.cs
P-14	В токене только id и логин, права — в BFF	роли в токене	смена прав действует без перевыпуска токенов	RBAC BFF с кешем	права-и-аудит.md §1–2
P-15	RBAC «группа × ресурс × маска действий», дерево групп	4 фиксированные роли в коде; варианты 1, 3, 4 концепта	новые ресурсы и роли — данными, без выката; вложенность групп	access_grants, group_closure	01. Авторизация 1.3, 02 2.6

№	Решение	Отброшено	Почему	Итоговый метод	Где
P-16	Область видимости по дереву объектов — задел, не реализована	реализовать сразу	нужна проверка во всех сервисах, иначе либо ничего не меняет, либо отрезает инженеров от данных	исключение — <code>/readings/score</code> для «Логов»	домены-и-сущности.md §13.8
P-17	Открытый ключ отдаётся PEM по <code>/.well-known/jwks</code>	стандартный JSON JWKS	так сделано в tf-auth; потребители (BFF, tfkit, воронка) приведены к PEM	PEM, кеш 1 ч	BFF DECISIONS.md «AUTH_JWKS_URL отдаёт PEM»
P-18	Техулётки через <code>score</code> в токене — концепт, временно список <code>sub</code>	долгие токены без прав	принимающему сервису не нужна своя база прав	<code>TF_*_SERVICE_SUBS</code> в Vault до появления <code>score</code>	права-и-аудит.md §2
P-19	LDAP/AD — контроллер домена в инфраструктуре, вход через каталог — следующий шаг	полная интеграция на хакатоне	каталога заказчика нет	Samba AD в <code>think-infra/samba</code> , схема учёток «вариант B»	01. Авторизация 1.7
P-20	Секреты только в Vault, у каждого сервиса своя роль	<code>.env</code> на серверах; GitHub Secrets; Vault в dev-режиме (в памяти)	утечки через git и файлы; dev-Vault терял секреты при перезапуске	Vault raft, AppRole <code>tf-svc-*</code> , распечатывание 2 из 3 ключей	hashicorp/RE ADME.md
P-21	Три способа доставки секрета в контейнер	единый entrypoint для всех	.NET-сервисы не меняют код (entrypoint), Python/Go-сервисы ходят в Vault сами, сервисы инфраструктуры получают переменные при выкатке	<code>deploy.sh</code> , <code>vault-entrypoint.sh</code> , <code>tfkit/kit.go</code>	SYSTEM.md 2.3
P-22	Веб-интерфейс Vault — поддомен через nginx	открыть порт Vault; оставить только SSH-туннель	не перезапускать Vault (запечатается) и не менять выданные <code>role_id</code> ; TLS снимает nginx	<code>https://vault.greefob.ru</code> , лимит входа	web-server/nginx/vault.conf.template
P-23	Наружу только nginx; лимиты и allowlist на nginx	лимиты в каждом сервисе	единая точка защиты: размер тела, частота, IP шины	<code>client_max_body_size 8m</code> , <code>limit_req</code> , <code>FUNNEL_EVENTS_ALLOW</code>	nginx.conf.template

3.3. Аудит

№	Решение	Отброшено	Почему	Итоговый метод	Где
P-24	Два журнала: действий и запросов	один журнал на всё	разная ценность и срок хранения (год против 90 дней)	<code>audit.events</code> , <code>audit.requests</code>	права-и-аудит.md §6.1
P-25	Доставка через поток Redis	HTTP POST <code>/audit/events</code>	сервис не ждёт аудит; если аудит лежит, события ждут в потоке	<code>XADD audit</code> , группа <code>audit-writer</code>	docs/backend/audit/audit.py

№	Решение	Отброшено	Почему	Итоговый метод	Где
P-26	Журнал неизменяем на уровне базы	только права роли	триггеры запрещают UPDATE / DELETE / TRUNCATE всем, включая владельца	<code>audit.forbid_change()</code>	docs/backend/audit/schema.sql
P-27	Протухший токен не пишется в журнал действий	писать каждый отказ	обновление раз в 10 минут у каждого пользователя забило бы журнал	только строка 401 в журнале запросов	права-и-аудит.md §6.2
P-28	Хэш-цепочка записей не делается	цепочка для доказательства неизменности	для хакатона избыточна; неизменность дают триггеры	—	там же, §6.6

3.4. Данные и разметка

№	Решение	Отброшено	Почему	Итоговый метод	Где
P-29	Разметка восстанавливается по журналу	ждать журнал ОДС и реестр оборудования	организаторы их не дадут (ответы 12–13)	эпизод = сработки одного типа с паузой ≤ 1 ч; шум размечается отдельно	ML/pipeline/labels.py, results/labels.md
P-30	2021 год исключён	обучать на всех семи годах	две системы мониторинга работали параллельно (ответ 6); семь лет вместо трёх дают $\pm 0,005$	обучение 2022–2024, проверка 2025, тест 2026	analytics.md §4
P-31	Флаг «тревожное» не используется	брать как метку	его смысл меняется по годам: 7 % у «Обнаружены» в 2023-м, 100 % в 2026-м	метки по справочнику состояний	документация.md §4.1
P-32	Погода не входит в модель	Open-Meteo: осадки, давление, влажность, снег, грунт, расход реки	11 пар моделей из 12 не лучше без погоды	—	analytics.md §1, §2, §5
P-33	Загазованность — шестой тип сверх ТЗ	слить с пожаром	своя реакция (вентиляция, а не пожарный расчёт); 600 эпизодов испортили бы пожар	6 типов прогноза	документация.md §1.1
P-34	Карта — сгенерированные линейные объекты по пикетам + развёртка коллектора	реальные координаты	московская система координат наружу не выдаётся (ответ 19)	пикет = 10 м, слои карты в BFF	домены-и-сущности.md §13.2–13.3

3.5. Модель

№	Решение	Отброшено	Почему	Итоговый метод	Где
P-35	Мера выбора — ложные часы при равной полноте	F1, ROC-AUC, PR-AUC, число сигналов	тревога горит часами; PR-AUC этого не видит, число сигналов вознаграждает гладкую оценку	сравнение в ложных часах + свежие поимки	analytics.md §34, §40

№	Решение	Отброшено	Почему	Итоговый метод	Где
P-36	Своя модель каждому типу, 5 зёрен	одна модель на все типы; LightGBM из плана	одно зерно меняет результат до 2,5 раза	CatBoost ×5 (пожар, газ, подтопление, проникновение), XGBoost ×5 (датчик), смесь CatBoost + TCN (оборудование)	документация. md §4.5
P-37	Нейросеть TCN только у отказа оборудования	сеть у всех типов; окна 336 и 72 ч; ежемесячное переобучение сети	у остальных типов вровень или хуже бустинга; переобучение теряет свежие поимки	4 версии оборудования для главного диспетчера	документация. md §5.5–5.6
P-38	Порог — скользящая доля часов под тревогой	фиксированный порог по F1; пороги по объектам	фиксированный промахивается в 0–4,4 раза; пороги по объектам дают втрое больше ложных	квантиль оценок парка за 90 суток на долю share	ML/settings/operating.md
P-39	Канал «по факту» рядом с прогнозом	только прогноз	прогноз теряет часть эпизодов; вместе с фактами порог поднимается, ложных на 65–85 % меньше	объявление на 10-й минуте эпизода, полнота 99,8 %	analytics.md §10, §15
P-40	Настройки таблицами главного диспетчера, а не переобучением	переобучать при каждой смене условий	переобучение по T3 — по согласованию (§8); таблицы действуют со следующего часа	доли и reject_k, график работ, игнорируемые периоды, версия типа	документация. md §6.3
P-41	Дообучение выключено по умолчанию	дообучение поверх прошлой модели (warm); ежемесячное	warm даёт втрое больше ложных; признаки не дрейфуют чаще раза в 150 суток	retrain.request при TF_MODEL_RETRAIN=on, скользящее окно по типам	analytics.md §13, §19
P-42	Молчание по графику работ	учить модель на отметке работ; подавлять по косвенному признаку	график снимает 45–59 % ложных часов газа без потерь	MUTED с причиной и ссылкой на строку графика	analytics.md §62
P-43	Правило отклонения только у газа и подтопления	у всех типов	у остальных добавляет пропусков больше, чем убирает ложных	reject_k = 0,2 у газа и подтопления	analytics.md §43–46
P-44	Уверенность — изотоническая калибровка	конформные предсказания; сырая оценка	конформные отброшены с обоснованием; калиброванная доля читается буквально	confidence из calibration.json	analytics.md §6, §17
P-45	Рекомендации — словарь правил, не модель	генерировать моделью	прозрачность, номер правила и обоснование у каждой меры	70 правил, режимы «прогноз» и «по факту»	analytics.md §61
P-46	Срочный прогноз на 6 ч не ставится	второй контур на 6 ч	прибавка меньше 5 % или выгоднее поднять долю суточного	суточный горизонт 24 ч	документация. md §5.6

№	Решение	Отброшено	Почему	Итоговый метод	Где
P-47	Сглаживание, предел длины тревоги, вторая ступень — не ставятся	ставить	проверены, не окупаются	склейка тревог 6 ч	документация. md §4.7

3.6. Инфраструктура и выкатка

№	Решение	Отброшено	Почему	Итоговый метод	Где
P-48	Стенд = ветка, выкатка из GitHub Actions на self-hosted runner	ручные docker compose на сервере	воспроизводимость и ревью	deploy- <сервис>.yml → scripts/deploy.sh	scripts/deploy.sh
P-49	Образы не собираются на слабом dev-сервере	сборка по SSH на сервере	сборка Go-образа исчерпала память и положила стенд 27.09	сборка в CI, доставка docker save /Docker Hub	SYSTEM. md, кейс 1
P-50	Почта prod — Resend на порту 2587	Gmail SMTP 587/465; HTTP API почтового сервиса	хостинг prod режет 25/465/587; 2587 открыт, код не меняется	smtp.resend.com:2587, MAIL_FROM=noreply@thinkfaster.ru	mailing/README. md
P-51	Сертификат Let's Encrypt с автопродлением, поддомены — в тот же сертификат	отдельные сертификаты	одна процедура продления	certbot webroot, DOMAIN_ALIASES , VAULT_DOMAIN	web-server/
P-52	Эмулятор на dev, на prod — никогда	эмулятор только локально	на dev нет шины; на prod он записал бы тестовые данные в prod-Kafka	TF_FUNNEL_PULL только на dev	SYSTEM. md, tf-emulator
P-53	Один путь выкатки каждого сервиса	воронка и модель из двух репозиторий с разными параметрами	конфликтующие параметры пересоздавали бы контейнер	воронка и модель — только из Think-Faster	deploy-apps-prod.yml